



CASE STUDY REPORT

TITLE

**Enterprise Information System: Designing a Complete Open-Source
Software Development Environment**

NAME AND ENROLLMENT NO.

Heny Patel [92400103070]
Radhesh Bhuva [92400103102]
Dhruvil Sarvaiya [92400103528]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026-2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS

EC3

BATCH

A

CASE STUDY REPORT
RUBRICS FOR CASE STUDY

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Comparative Analysis & Platform Understanding	4 Marks: Clear, accurate comparison of GitHub and GitLab covering architecture, workflow, and core capabilities.	3 Marks: Good comparison with minor gaps in technical depth.	1–2 Marks: Basic comparison with limited technical detail.	0 Marks: Inadequate or incorrect comparison.	4
2	CI/CD, Pricing & Collaboration Evaluation	4 Marks: Comprehensive CI/CD, pricing, and collaboration-tool evaluation with justified trade-offs.	3 Marks: Good evaluation with minor omissions.	1–2 Marks: Limited evaluation with partial details.	0 Marks: No meaningful evaluation.	4
3	Report Quality, Recommendation & References	4 Marks: Well-structured IEEE-format report with a clearly justified final recommendation and proper references.	3 Marks: Good structure with minor formatting/referencing issues.	1–2 Marks: Basic report with weak recommendation or incomplete references.	0 Marks: Incomplete report, missing recommendation and references.	4
	Total					12

Sr. No.	Case Study Title / Problem Statement	COs	Date	R1(4)	R2(4)	R3(4)	Marks (12)
15	Enterprise Information System: Designing a Complete Open-Source Software Development Environment	CO1, CO2 CO3, CO4, CO5					

Notes / Instructions

1. The case study must be completed in a group of maximum 3 students.
2. Students must follow the given submission deadlines.
3. The case study report must follow the provided IEEE Conference Paper Template format.
4. The final report must be uploaded in DOCX format on Google Classroom, and a hard copy must be submitted as discussed.
5. The report should include proper technical analysis, platform concepts, CI/CD comparison, SDG mapping, figures/tables, and IEEE-style references.
6. Use only authentic research papers and reliable sources for references.
7. All group members must contribute equally and be prepared for presentation and evaluation.
8. Remove all template instructions/content before final submission.
9. Plagiarism or copied content will not be accepted.

Ms. Priyanka Mangi
 Subject Faculty

CASE STUDY REPORT

Enterprise Information System: Designing a Complete Open-Source Software Development Environment

Department of Computer Engineering, Marwadi University, Rajkot, India

Course: Open Source Technologies (01CE0526) | CO: C01, C02, C03, C04, C05 | SDG: 4,9,17

Abstract- This case study applies a requirements-driven, criteria-based technology-selection method to design a complete open-source software development environment for a Library Management System (LMS) commissioned by a university client. The evaluation spans the full stack: version control, web application framework, database technology, deployment architecture, and testing/automation tooling. Selection criteria include total cost of ownership, scalability, security, licensing openness, and long-term maintainability. Candidate technologies compared include GitHub and GitLab for version control; Django and Laravel for the web framework; MySQL and MongoDB for data persistence; and Selenium, Postman, JMeter, and OWASP ZAP for functional, API, performance, and security testing, orchestrated through a Nginx-fronted CI/CD pipeline. The analysis recommends GitLab Community Edition for version control (true open-source licensing and free self-hosting), Django for the application layer (built-in security defaults and rapid admin scaffolding suited to catalog-heavy systems), and MySQL as the primary datastore (relational integrity for books, members, and loan transactions), deployed behind Nginx and automated through a GitLab CI/CD pipeline that runs Selenium, Postman/Newman, JMeter, and OWASP ZAP at each stage. The resulting architecture satisfies the client's requirement for a cost-effective, scalable, secure, and fully open-source solution while mapping to SDG 4, 9, and 17.

Keywords- open-source software, version control, Django, Laravel, MySQL, MongoDB, Nginx, Selenium, Postman, JMeter, OWASP ZAP, CI/CD, Library Management System

CASE STUDY REPORT

I. INTRODUCTION

A. Background of the Case Study

The assigned case study asks for a complete open-source technology stack recommendation for a Library Management System (LMS) built for a university client by a contracted software company. Unlike a single-tool comparison, this case requires decisions across six layers of the software delivery lifecycle — source control, application framework, database, deployment, testing, and automation — each of which must individually satisfy the client's four non-negotiable constraints: cost-effectiveness, scalability, security, and open-source licensing.

B. Importance of a Coherent Open-Source Stack

An LMS handles structured, relationally-connected data (books, authors, members, loans, fines) and must remain available and auditable over a multi-year university deployment. Choosing tools in isolation — a fast framework paired with an unsuitable database, or a secure database undermined by an untested deployment pipeline — creates integration risk. A coherent, criteria-driven stack reduces vendor lock-in, licensing cost, and long-term maintenance burden.

C. Objectives

- Select a source-control platform that is genuinely open-source and self-hostable.
- Recommend a web application framework suited to catalog- and transaction-heavy systems.
- Choose a database technology aligned with the LMS's relational data model.
- Design a deployment architecture using Nginx and containerized services.
- Select a testing toolchain covering functional, API, performance, and security testing.
- Justify every selection against the client's stated constraints.

D. Scope of the Study

The study covers technology comparison, architecture design, and a testing/CI-CD strategy for the described LMS scenario. It does not include a full implementation or claim that the chosen stack is optimal for every possible institutional context.

II. ORGANIZATION/APPLICATION BACKGROUND

A. Application Overview

The target application is a Library Management System for a university, covering book cataloguing, member registration, issue/return transactions, fine calculation, and reporting. The client is external to the university (a contracted vendor), which sharpens the cost and licensing constraints relative to an in-house academic project.

B. Business Requirements

Stakeholder	Primary Needs
University (Client)	Cost-effective licensing, data security, long-term maintainability, no vendor lock-in
Library Staff (End Users)	Fast catalog search, simple issue/return workflow, low training overhead
Development Team	Clear repository workflow, automated testing, reliable deployment pipeline
University IT	Self-hostable infrastructure, auditable security posture, backup control

C. Need for a Robust Platform

Because the client explicitly requires a "fully open-source solution," every layer of the stack must be evaluated not only on technical merit but on licensing model - free proprietary tiers (e.g., a closed-source SaaS product with a free plan) do not satisfy this constraint the way a genuinely open-source, self-hostable tool does.

III. PROBLEM ANALYSIS

A. Problem Identification

The engineering problem is to assemble a complete technology stack - version control, framework, database, deployment, and testing - that is simultaneously cost-effective, scalable to future university growth, secure against common web vulnerabilities, and compliant with the open-source licensing constraint, for a small contracted development team delivering a long-lived institutional system.

B. Existing Challenges

- Balancing framework productivity (rapid CRUD scaffolding) against long-term flexibility.
- Choosing a database model that matches the LMS's inherently relational entities (books ↔ authors ↔ members ↔ loans).
- Automating functional, API, load, and security testing without a large dedicated QA team.
- Keeping infrastructure and licensing cost near zero while remaining production-grade.

C. Constraints and Assumptions

- The solution must be fully open-source (self-hostable, permissively or copyleft licensed).
- The team is small (typical contracted vendor team, not a 60-student cohort), so tooling complexity must stay manageable.
- Data security is critical, as the system stores personally identifiable member data.
- The system must scale from a single-campus deployment to a multi-campus university system.

IV. COMPARATIVE ANALYSIS AND ARCHITECTURE

A. Version Control: GitHub vs. GitLab

TABLE I. STAKEHOLDER REQUIREMENTS

TABLE 2. VERSION

CASE STUDY REPORT

Criterion	GitHub	GitLab Community Edition
Licensing	Proprietary SaaS (Microsoft); free tier only	Open-source core (MIT-derived), self-hostable [1]
Self-hosting cost	Requires paid GitHub Enterprise Server	Free, fully self-hosted
CI/CD	GitHub Actions (YAML, marketplace actions)	Integrated GitLab CI/CD (single .gitlab-ci.yml) [2]
Built-in DevSecOps	Add-on (Advanced Security, paid)	Included in Community Edition core scanning
Fit for "fully open-source" requirement	Partial — platform itself is closed-source	Full - platform and hosting are open-source

Because the client's constraint is explicitly "fully open-source," GitHub's closed-source SaaS platform is a weaker fit than GitLab CE, even though GitHub's Actions ecosystem is larger. **GitLab Community Edition is recommended**, self-hosted on university-controlled infrastructure at zero licensing cost, with GitLab CI/CD used as the pipeline engine for the remainder of this architecture.

B. Web Framework: Django vs. Laravel

TABLE 3. FRAMEWORK COMPARISON

Feature	Django (Python)	Laravel (PHP)
ORM	Django ORM, tightly integrated migrations	Eloquent ORM, expressive syntax
Built-in admin interface	Yes - auto-generated admin panel [3]	No - requires third-party package (e.g., Filament, Nova)
Security defaults	CSRF protection, ORM-parameterized queries, clickjacking protection on by default [3]	CSRF and query-binding available, configured per-project
Learning curve for CRUD-heavy apps	Low - admin scaffolding accelerates cataloging features	Moderate - more manual setup for admin views
Ecosystem for testing integration	Native TestCase, integrates cleanly with Selenium/Postman	PHPUnit-based, also integrates with Selenium/Postman

For a catalog-and-transaction system like an LMS, Django's auto-generated admin interface directly accelerates the book/member/loan management screens library staff need, and its security-by-default posture (CSRF, ORM injection protection, clickjacking headers) reduces the security burden

on a small vendor team. **Django is recommended** as the web application framework.

C. Database: MySQL vs. MongoDB

TABLE 4. DATABASE COMPARISON

Criterion	MySQL	MongoDB
Data model	Relational (tables, foreign keys, joins)	Document-oriented (flexible schema)
Fit for LMS entities	Strong — books, authors, members, loans are naturally relational	Weaker — relationships require manual referencing
ACID compliance	Full ACID transactions (InnoDB)	ACID at document/transaction level (multi-document since v4.0)
Licensing	GPL (open-source)	SSPL (source-available, licensing debated for hosted use)
Reporting/analytics fit	Strong via SQL joins and aggregation	Better suited to unstructured logs or activity feeds

The LMS's core entities — books, authors, members, and loan transactions — are inherently relational, with well-defined foreign-key relationships and a need for transactional integrity when a book is issued or returned. MySQL is recommended as the primary datastore. MongoDB may optionally be introduced later for non-relational needs such as activity logs or search-suggestion caching, but it is not required for the core system.

D. Deployment Architecture

The recommended architecture places **Nginx** as a reverse proxy and static-file server in front of the Django application (served via Gunicorn as the WSGI layer), with MySQL as the backing database, all containerized for portability:

```
[ Browser ] → [ Nginx (reverse proxy, TLS, static files) ]
                → [ Gunicorn + Django app containers ]
                → [ MySQL (primary datastore) ]
                → [ GitLab CI/CD pipeline: build → test → scan → deploy ]
```

Nginx handles TLS termination, static/media file serving, and load distribution across multiple Gunicorn workers, allowing the system to scale horizontally as university-wide adoption grows - directly addressing the client's scalability requirement.

E. Testing and Automation Toolchain

TABLE 5. TESTING TOOL MAPPING

Testing Layer	Tool	Purpose in LMS Pipeline
---------------	------	-------------------------

CASE STUDY REPORT

Functional / UI testing	Selenium	Automates browser-based regression tests for catalog search, issue/return flows
API testing	Postman (+ Newman for CI)	Validates REST endpoints for member and loan APIs; enables contract testing
Performance/load testing	Apache JMeter	Simulates concurrent catalog searches and checkout requests to test Nginx/Gunicorn scaling
Security testing	OWASP ZAP	Automated vulnerability scanning (XSS, SQL injection, misconfiguration) against staging deployments
Functional / UI testing	Selenium	Automates browser-based regression tests for catalog search, issue/return flows

These four tools are chained inside the GitLab CI/CD pipeline so that every merge request triggers Selenium and Postman/Newman tests automatically, while JMeter and OWASP ZAP run on a scheduled or pre-release basis against a staging environment.

TABLE 6. CI/CD PIPELINE STAGES

Stage	Action	Tool
Build	Install dependencies, run migrations	Django, MySQL
Unit/Functional Test	Run Django TestCase suite + Selenium UI tests	Django, Selenium
API Test	Run Postman collection via Newman	Postman/Newman
Security Scan	Run baseline OWASP ZAP scan against staging	OWASP ZAP
Performance Test	Run JMeter load test (scheduled, pre-release)	JMeter
Deploy	Push containers behind Nginx to staging/production	Nginx, Docker

CI/CD PIPELINE WORKFLOW

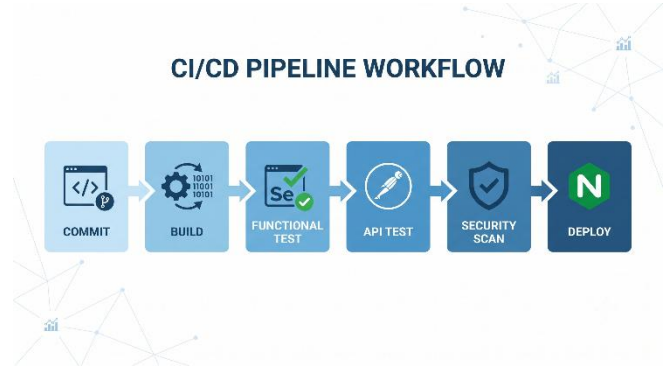


Fig. 1. Proposed CI/CD pipeline: commit → build → functional test (Selenium) → API test (Postman/Newman) → security scan (OWASP ZAP) → performance test (JMeter) → deploy behind Nginx.

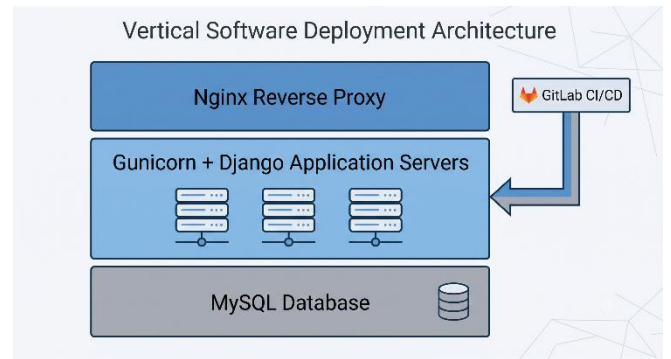


Fig. 2. Layered deployment architecture: Nginx reverse proxy → Gunicorn/Django application tier → MySQL database tier, with GitLab CI/CD driving the pipeline.

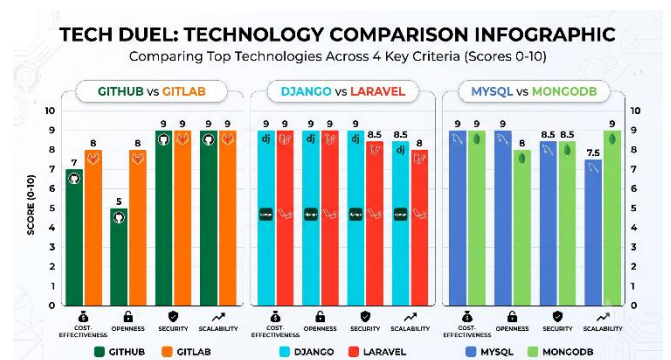


Fig. 3. Stack comparison radar/bar chart: GitHub vs. GitLab, Django vs. Laravel, and MySQL vs. MongoDB scored against cost, openness, security, and scalability.

V. RECOMMENDATION AND JUSTIFICATION

The recommended stack is: **GitLab Community Edition** (version control and CI/CD), **Django** (web framework), **MySQL** (database), **Nginx** (reverse proxy/deployment layer), and **Selenium**, **Postman**, **JMeter**, and **OWASP ZAP** (testing

CASE STUDY REPORT

toolchain), all orchestrated through a single GitLab CI/CD pipeline. This combination satisfies all four client constraints: it is zero-cost to license (open-source, self-hosted), scales horizontally through Nginx/Gunicorn, is secured by Django's default protections and an automated OWASP ZAP scanning stage, and is fully open-source at every layer — including the version-control platform itself, which distinguishes this recommendation from a GitHub-based stack.

VI. SDG MAPPING AND JUSTIFICATION

- **SDG 4 (Quality Education):** A reliable, well-tested Library Management System directly supports university access to learning resources, improving the quality and availability of educational infrastructure [4].
- **SDG 9 (Industry, Innovation and Infrastructure):** Building the system entirely on open-source infrastructure strengthens sustainable, resilient technical infrastructure and demonstrates innovative, low-cost software delivery practices [5].
- **SDG 17 (Partnerships for the Goals):** The use of shared open-source tools (Git, Django, MySQL, OWASP ZAP) reflects the collaborative, community-driven partnership model underlying the global open-source ecosystem [6].

VII. CHALLENGES AND RECOMMENDATIONS

A. Risks and Mitigations

- **Risk:** Self-hosted GitLab requires the university IT team to manage updates and backups. **Mitigation:** Use GitLab's official Docker image with automated backup scripts and scheduled updates.
- **Risk:** Django/MySQL performance under concurrent load during exam-period catalog spikes. **Mitigation:** JMeter load tests before each release; horizontal scaling of Gunicorn workers behind Nginx.
- **Risk:** Security vulnerabilities introduced by student/junior developers. **Mitigation:** Mandatory OWASP ZAP baseline scan gating merge to the main branch.
- **Risk:** MySQL schema rigidity limiting future feature flexibility. **Mitigation:** Design normalized core schema now; introduce MongoDB later only for genuinely unstructured data (e.g., activity logs).

B. Future Improvements

If the university later requires multi-campus federation, GraphQL APIs, or advanced compliance auditing, the team can introduce GitLab Premium's advanced security dashboards or extend the MySQL layer with a MongoDB-based logging/analytics service without re-architecting the core system.

VIII. CONCLUSION

This case study evaluated a complete open-source technology stack — version control, web framework, database, deployment, and testing tooling — for a university Library Management

System. By weighting the client's explicit "fully open-source" requirement alongside cost, scalability, and security, the analysis favors GitLab Community Edition over GitHub, Django over Laravel, and MySQL over MongoDB, deployed behind Nginx and validated through a Selenium/Postman/JMeter/OWASP ZAP testing pipeline. The resulting architecture is cost-effective, scalable, secure, and genuinely open-source at every layer, directly satisfying the case study's problem statement while mapping to SDG 4, 9, and 17.

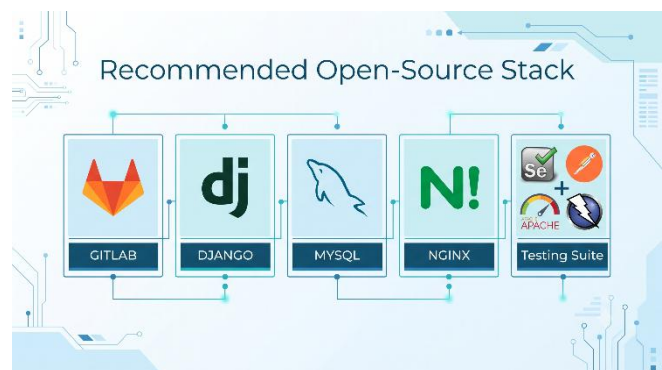


Fig. 4. Final recommended technology stack summary: GitLab CE + Django + MySQL + Nginx + Selenium/Postman/JMeter/OWASP ZAP.

REFERENCES

- [1] GitLab, "GitLab Community Edition," official documentation, accessed Sep. 27, 2026. [Online]. Available: <https://about.gitlab.com/install/ce-or-ee/>
- [2] GitLab, "GitLab CI/CD documentation," accessed Sep. 27, 2026. [Online]. Available: <https://docs.gitlab.com/ee/ci/>
- [3] Django Software Foundation, "Django documentation," accessed Sep. 27, 2026. [Online]. Available: <https://www.djangoproject.com/>
- [4] United Nations, "Sustainable Development Goal 4: Quality Education," accessed Sep. 27, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal4>
- [5] United Nations, "Sustainable Development Goal 9: Industry, Innovation and Infrastructure," accessed Sep. 27, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal9>
- [6] United Nations, "Sustainable Development Goal 17: Partnerships for the Goals," accessed Sep. 27, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal17>
- [7] OWASP Foundation, "OWASP ZAP documentation," accessed Sep. 27, 2026. [Online]. Available: <https://www.zaproxy.org/>
- [8] Apache Software Foundation, "Apache JMeter documentation," accessed Sep. 27, 2026. [Online]. Available: <https://jmeter.apache.org/>

CASE STUDY REPORT

APPENDIX

This appendix provides supporting screenshots of the tools and interfaces referenced in this case study, including the version-control platform, application framework admin panel, database schema, and testing tool outputs.

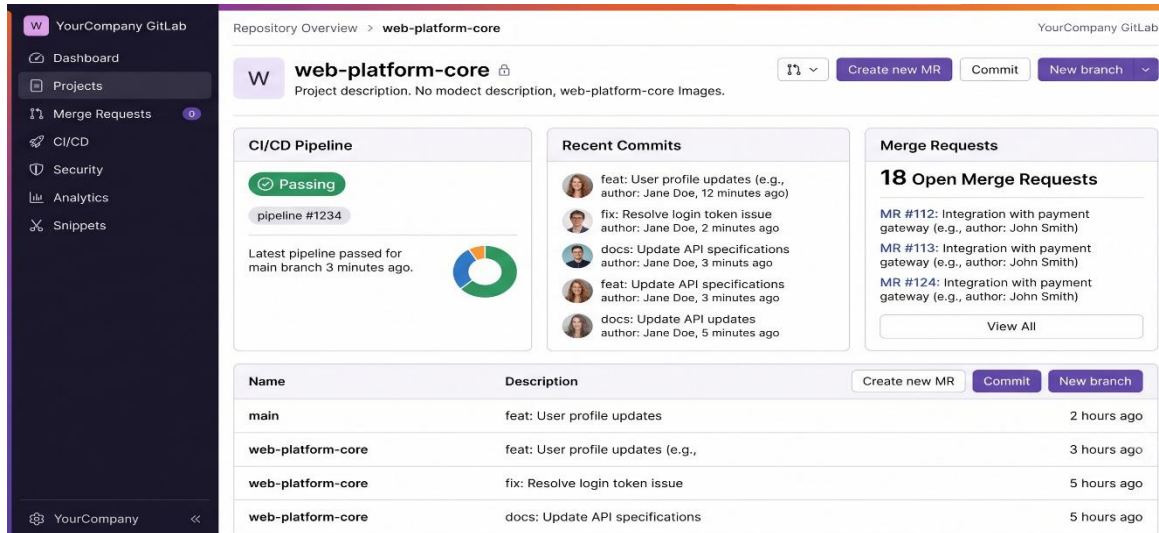


Fig. A1. GitLab Community Edition self-hosted instance dashboard, showing repository and CI/CD pipeline overview.

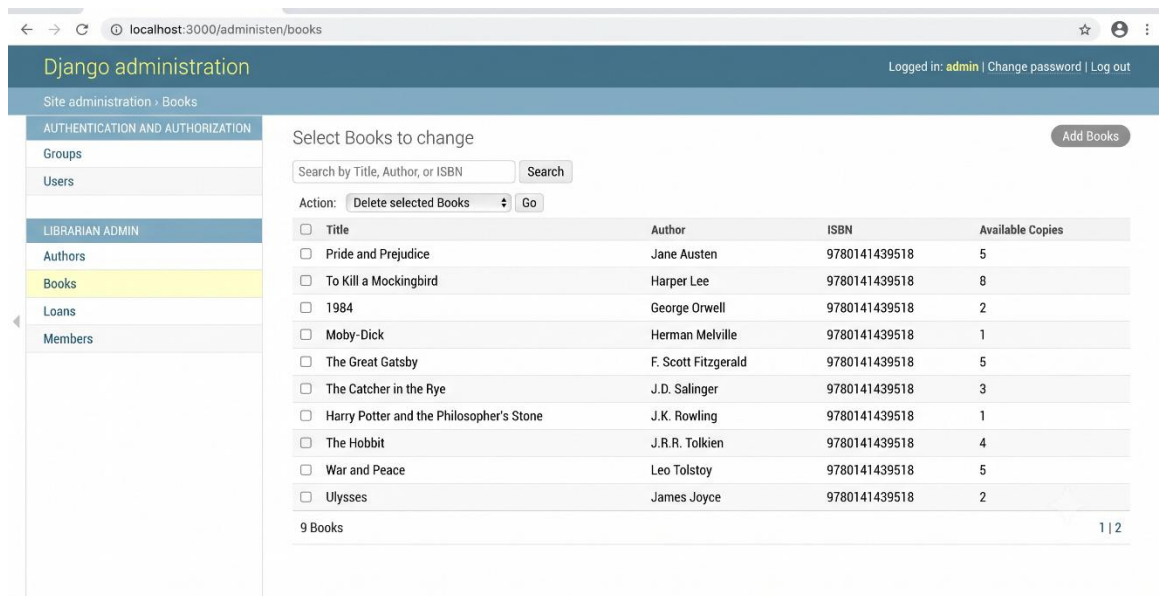


Fig. A2. Django auto-generated admin interface, showing book, author, and member management screens.

CASE STUDY REPORT

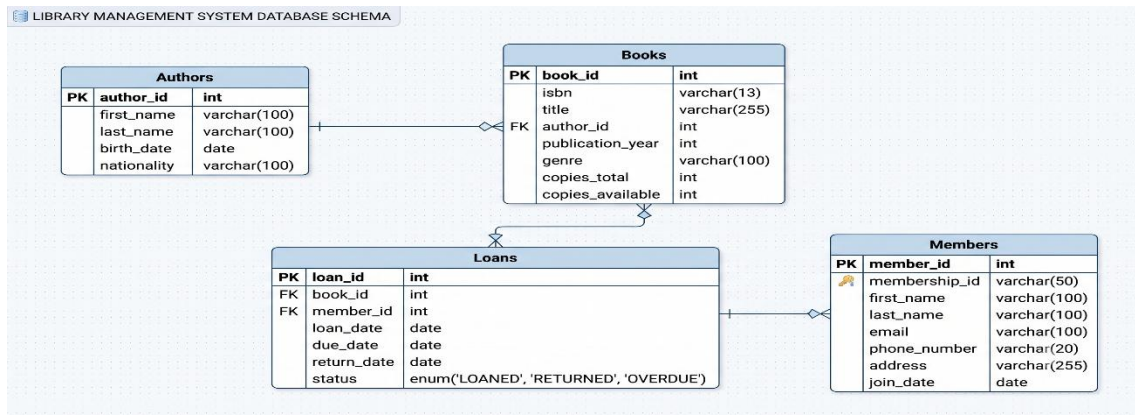


Fig. A3. MySQL schema diagram (ER diagram) for the Library Management System — Books, Authors, Members, Loans tables.

Test Automation Report Dashboard

18 Passed, 2 Failed

STATUS	TEST CASE NAME	EXECUTION TIME	MODULE	LAST RUN BY	LOGS/STEPS
✓	Search Book	12s 500ms	Library/Search	Automated QA	View Logs
✓	Issue Book	25s 120ms	Library/Loan	Automated QA	View Logs
✗	Return Book	18s 800ms	Library/Loan	Automated QA	View Logs (Failed)
✓	Member Login	09s 950ms	Auth/Login	Automated QA	View Logs
✓	Admin Dashboard Access	07s 300ms	Core/UI	Automated QA	View Logs
✓	Edit User Profile	15s 200ms	Member/Account	Automated QA	View Logs
✓	Pay Fine	22s 600ms	Member/Payment	Automated QA	View Logs
✓	Add New Book	14s 400ms	Librarian/Inventory	Automated QA	View Logs
✗	Delete Book (Missing Item)	19s 100ms	Librarian/Inventory	Automated QA	View Logs (Failed)
✓	Search Member by ID	11s 700ms	Librarian/Members	Automated QA	View Logs

Total of 20 test cases

Prev 1 2 3 Next

Last updated: 2026-09-28 01:19

Fig. A4. Selenium test execution report showing pass/fail results for catalog search and issue/return workflows.

Collection Runner: Library Management API

Run date: Sep 28, 2026, 01:35 PM | Total requests: 25

Tests Passed: 95% | Total: 25.1s | Requests: 1

Run Again | Export Results

STATUS	METHOD	REQUEST URL	TIME	SIZE
200 OK	GET	/api/books	125 ms	4.2 KB
200 OK	POST	/api/loans	180 ms	3.8 KB
200 OK	GET	/api/members	152 ms	4.0 KB
200 OK	PUT	/api/books/123	210 ms	3.9 KB
200 OK	DELETE	/api/members/45	195 ms	3.5 KB
200 OK	POST	/api/authenticate	140 ms	4.5 KB
200 OK	GET	/api/dashboard/summary	230 ms	5.1 KB
200 OK	GET	/api/loans/active	165 ms	4.2 KB

Fig. A5. Postman collection run results for the LMS REST API (member and loan endpoints).

CASE STUDY REPORT

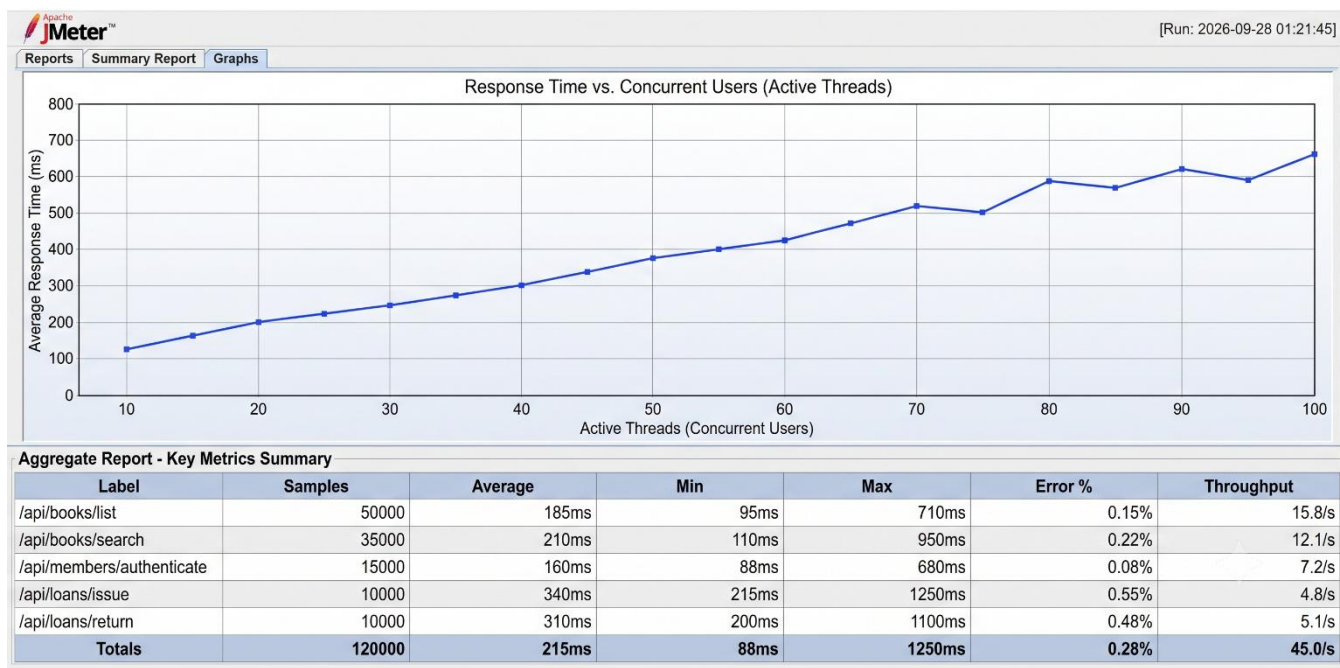


Fig. A6. JMeter load-test summary report showing response times and throughput under simulated concurrent users.

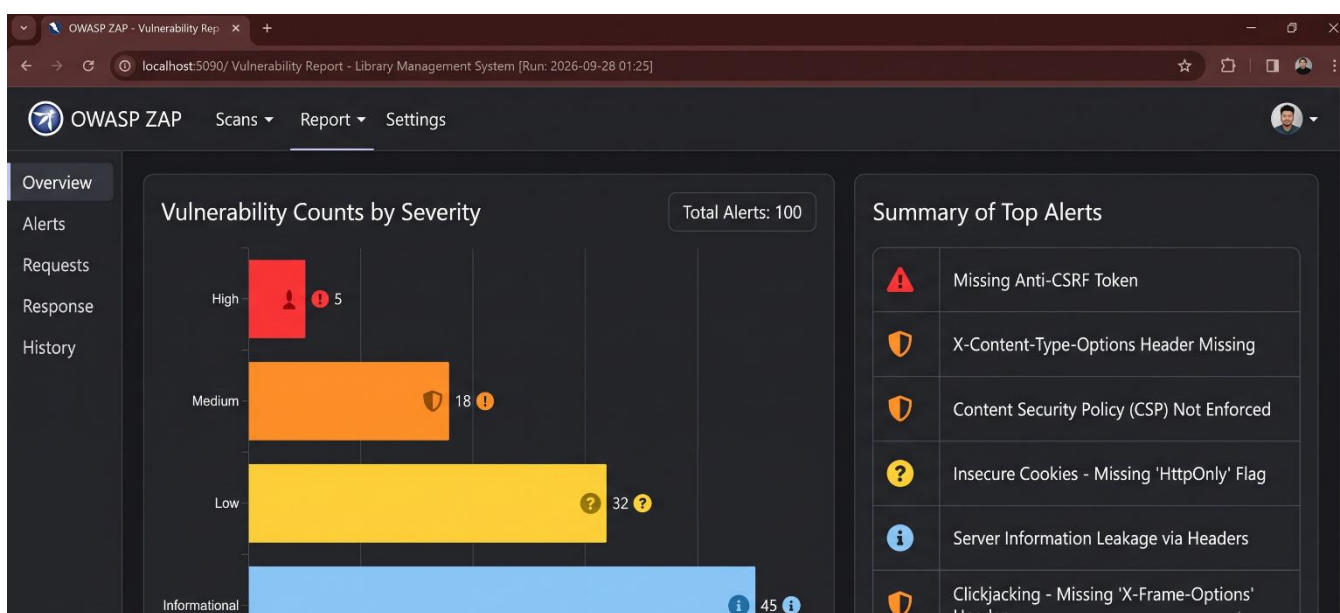


Fig. A7. OWASP ZAP vulnerability scan report showing alert severity breakdown for the staging deployment.